Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251230**
Nama Lengkap	**Okky Alexander**
Minggu ke / Materi	**06 / Struktur Kontrol Perulangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI DEFINISI PERULANGAN

Alur jalannya sebuah program dikendalikan oleh apa yang disebut sebagai struktur kontrol, yang dapat berupa urutan sekuensial, percabangan, perulangan, maupun kombinasi dari ketiganya. Struktur perulangan sendiri menjadi sangat penting ketika program dituntut untuk melakukan tugas yang sama secara berulang kali atau menyelesaikan proses bertahap yang memiliki pola langkah serupa. Selain itu, perulangan berfungsi sebagai sarana untuk mengakses dan mengolah sekumpulan data yang tersimpan dalam berbagai struktur data, seperti List, Tuple, Queue, hingga Stack. Di dalam ekosistem Python, terdapat tiga cara utama untuk mengimplementasikan perulangan, yaitu melalui perintah for, while, serta teknik rekursif, namun pembahasan kali ini akan difokuskan secara mendalam pada penggunaan for dan while.

MATERI BENTUK PERULANGAN FOR

Dalam pemrograman Python, struktur perulangan for umumnya digunakan ketika jumlah iterasi sudah diketahui secara pasti sejak awal. Sebagai contoh, saat kita perlu membaca 10 file teks secara berurutan, prosesnya tetap sama dari file pertama hingga terakhir meski isinya berbeda. Selain itu, for sangat efektif untuk menangani operasi pada rentang nilai tertentu, seperti menjumlahkan angka dari 1 hingga 100. Untuk mempermudah pembuatan rentang tersebut, Python menyediakan fungsi range(). Fungsi ini memiliki dua bentuk utama: range(stop) untuk menghasilkan urutan angka dari 0 hingga sebelum angka *stop*, serta range(start, stop, step) untuk menentukan titik awal, titik akhir (eksklusif), dan besarnya kenaikan atau penurunan nilai.

Secara teknis, perulangan ini menggunakan variabel (sering disebut *counter*) untuk menyimpan nilai yang sedang berjalan. Namun, jika kita hanya ingin mengulang suatu perintah tanpa mempedulikan urutan angkanya—seperti mencetak tulisan "Hello World" sebanyak 100 kali—kita bisa menggunakan tanda garis bawah (_) sebagai pengganti variabel *counter*. Menariknya, fungsi range() juga mendukung *step* negatif, yang memungkinkan kita melakukan perulangan mundur, misalnya untuk menampilkan urutan bilangan genap dari besar ke kecil.

Contoh Implementasi Kode

1. Perulangan dengan Rentang Standar (1-100)

```
for i in range(1, 101):
    print(i)
```

2. Perulangan Tanpa Membutuhkan Nilai Counter

```
for _ in range(1, 101):
    print('Hello World')
```

3. Perulangan Bilangan Genap (Maju)

```
for i in range(2, 101, 2):
   print(i)
```

4. Perulangan Bilangan Genap (Mundur/Step Negatif)

```
for i in range(100, 1, -2):
   print(i)
```

MATERI BENTUK PERULANGAN WHILE

Berbeda dengan for, struktur perulangan while di Python lebih fleksibel karena digunakan saat kita tidak tahu pasti berapa kali sebuah aksi harus diulang. Perulangan ini akan terus berjalan selama kondisi yang ditentukan masih bernilai benar (True). Secara umum, strukturnya melibatkan titik awal (start), kondisi berhenti (stop), operasi yang dijalankan, dan perubahan nilai langkah (step) jika diperlukan.

Salah satu skenario terbaik penggunaan while adalah untuk validasi input. Misalnya, ketika kita meminta pengguna memasukkan bilangan genap, kita tidak bisa memprediksi berapa kali pengguna akan salah memasukkan bilangan ganjil. Program akan terus "menagih" input hingga syarat bilangan genap terpenuhi, sebuah situasi yang sulit ditangani dengan for karena jumlah percobaannya tidak terhingga sampai kondisi sukses tercapai.

Contoh Implementasi Kode while

```
bilangan = 0
genap = False
while genap == False:
   bilangan = int(input('Masukkan bilangan genap: '))
   if bilangan % 2 == 0:
```

```
genap = True

print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

MATERI PENGGUNAAN BREAK DAN CONTINUE

Selain menggunakan kondisi berhenti yang standar, Python menyediakan perintah khusus untuk mengendalikan jalannya iterasi secara lebih presisi, yaitu melalui instruksi break dan continue. Kedua perintah ini memiliki peran yang bertolak belakang dalam memengaruhi siklus perulangan yang sedang berjalan.

Secara sederhana, break berfungsi sebagai tombol darurat yang menghentikan seluruh proses perulangan seketika, bahkan sebelum batas akhir perulangan tercapai. Ketika program menemui instruksi ini, ia akan langsung keluar dari blok perulangan dan melanjutkan eksekusi ke baris kode setelah perulangan tersebut. Sebaliknya, continue hanya berfungsi untuk membatalkan atau melompati iterasi yang sedang berlangsung saat itu juga. Alih-alih menghentikan seluruh proses, continue memaksa program untuk langsung melompat ke tahap atau angka berikutnya dalam antrean perulangan, sehingga instruksi yang ada di bawahnya pada iterasi tersebut akan diabaikan.

Implementasi Kode break dan continue

Berikut adalah perbandingan penerapan keduanya dalam kode program:

1. Menghentikan Perulangan dengan break Program ini seharusnya berjalan hingga angka 10, namun kita paksa berhenti total saat mencapai angka 5.

```
for i in range(1, 11):
    if i == 5:
        break
    else:
        print(i)
print('Selesai')
# Output: 1, 2, 3, 4, Selesai
```

2. Melompati Iterasi dengan continue Program ini tetap berjalan hingga angka 10, namun khusus saat angka 6, instruksi cetak dilewati.

```python
for i in range(1, 11):
    if i == 6:
        continue
    else:
        print(i)

print('Selesai')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\Koolyeah\github\PrakAlPro-71251230> python -u "e:\Koolyeah\github\PrakAlPro-71251230\tempCodeRunnerFile.python"
1
2
3
4
5
7
8
9
10
Selesai
PS E:\Koolyeah\github\PrakAlPro-71251230>
```

Gambar 6.1: contoh melompati di perulangan menggunakan continue.

MATERI KONVERSI DARI BENTUK FOR MENJADI BENTUK WHILE

Sebagian besar struktur perulangan for dalam Python dapat dikonversi ke dalam bentuk while karena keduanya berbagi logika dasar yang sama. Untuk melakukan konversi ini, kita harus mengidentifikasi tiga elemen kunci: nilai awal untuk memulai iterasi, nilai akhir sebagai batasan atau kondisi berhenti, serta langkah (step) yang memastikan perulangan terus bergerak maju hingga mencapai batas tersebut. Jika salah satu elemen ini terlewatkan—terutama bagian langkah—program berisiko mengalami infinite loop atau perulangan tanpa henti.

Sebagai ilustrasi, jika kita memiliki perulangan for yang mencetak angka 1 sampai 10, kita tahu bahwa nilainya dimulai dari 1, berakhir di 10, dan bertambah 1 di setiap putaran. Saat diubah ke bentuk while, nilai awal didefinisikan sebagai variabel di luar perulangan, kondisi akhir diletakkan tepat setelah kata kunci while, dan penambahan nilai dilakukan secara manual di dalam blok kode perulangan. Dengan cara ini, kedua metode tersebut akan menghasilkan output yang identik namun dengan fleksibilitas penulisan yang berbeda.

Perbandingan Konversi Kode

Berikut adalah perbandingan langsung antara penulisan menggunakan for dan hasil konversinya ke while:

1. Menggunakan for (Otomatis)

```
for i in range(1, 11):

    print(i)
```

2. Menggunakan while (Manual)

```
i = 1
while i <= 10:
    print(i)
    i = i + 1

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS E:\Koolyeah\github\PrakAlPro-71251230> python -u "e:\Koolyeah\github\PrakAlPro-71251230\tempCodeRunnerFile.python"
1
2
3
4
5
6
7
8
9
10
```

Gambar 6.2: contoh program menggunakan while.

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 6.1

```
def perkalian(a,b):
    print(a,"x",b ,"= ", end="")
    for i in range(a):
        print(f"{b} ",end="")
        if i < a -1:
            print("+ ",end="")
    print(f"= {a*b}.")


perkalian(6,5)
perkalian(7,10)
```

Gambar 6.3: Program Fungsi Perkalian.

Fungsi bernama perkalian yang bertujuan untuk memvisualisasikan konsep perkalian sebagai penjumlahan berulang. Saat fungsi dipanggil, program pertama-tama akan mencetak format perkalian awal (misalnya "6 x 5 = ") tanpa berpindah baris. Selanjutnya, program menggunakan struktur kontrol perulangan for dengan rentang sebanyak nilai a untuk mencetak angka b secara berulang. Di dalam perulangan tersebut, terdapat logika percabangan if yang berfungsi untuk menyisipkan simbol tambah (+) di antara angka-angka tersebut, namun simbol ini tidak akan dicetak setelah angka terakhir (ketika indeks i mencapai a - 1). Setelah seluruh rangkaian penjumlahan selesai ditampilkan, program menutup baris tersebut dengan mencetak hasil akhir dari operasi a * b.

Pada bagian akhir kode, fungsi ini diuji dengan dua pemanggilan berbeda. Pemanggilan perkalian(6,5) akan menghasilkan tampilan penjumlahan angka 5 sebanyak 6 kali yang berjumlah 30, sedangkan perkalian(7,10) akan menampilkan penjumlahan angka 10 sebanyak 7 kali dengan hasil akhir 70. Kode ini sangat efektif untuk menjelaskan logika dasar matematika kepada pemula melalui representasi visual di terminal.

LATIHAN 6.2

```
def ganjil(bawah,atas):
    if bawah < atas:
        print(f"bawah = {bawah}, atas = {atas}. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: ",end="" )
        for i in range(atas):
            if i > bawah:
                if i % 2 != 0:
                    print(i,end="")
                    if i < atas -1:
                        print(" ",end="")

        print(f".")
    elif bawah > atas:
        print(f"bawah = {bawah}, atas = {atas}. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: ",end="" )
        for i in range(bawah, 0 , -1):
            if i > atas:
                if i % 2 !=0:
                    print(i,end="")
                    if i > atas+1:
                        print(" ",end="")
        print(f".")
ganjil(10,30)
ganjil(97,82)
```

Gambar 6.4: Program Fungsi Bilangan Ganjil.

Fungsi bernama ganjil yang dirancang untuk mencari dan menampilkan bilangan ganjil di antara dua batas angka, yaitu bawah dan atas. Fungsi ini memiliki dua alur logika utama yang diatur melalui percabangan if-elif. Jika nilai bawah lebih kecil dari atas, program akan melakukan perulangan maju untuk mencari angka ganjil yang lebih besar dari batas bawah hingga mendekati batas atas. Sebaliknya, jika nilai

bawah lebih besar dari atas, program akan melakukan perulangan mundur menggunakan *step* negatif -1 untuk mencari angka ganjil dari nilai besar menuju nilai kecil. Di dalam kedua proses tersebut, terdapat pengecekan menggunakan operator modulo (i % 2 != 0) untuk memastikan hanya angka ganjil yang dicetak, serta pengaturan spasi agar tampilan angka tidak berdempetan sebelum diakhiri dengan tanda titik.

Ketika kode tersebut dijalankan dengan perintah ganjil(10, 30), program akan mendeteksi urutan dari kecil ke besar dan menampilkan angka ganjil di antara 10 hingga 30, yaitu mulai dari 11 sampai 29. Sementara itu, pemanggilan ganjil(97, 82) akan memicu logika perulangan mundur yang menampilkan deretan angka ganjil dari 97 ke bawah hingga mencapai batas di atas 82, yaitu angka 97, 95, dan seterusnya hingga 83.

LATIHAN 6.3

```
def ips(x):
    total = 0
    bobot = 0
    for i in range(1,x+1):
        nilai = str(input(f"Nilai Mk {i}: "))
        if nilai == "A":
            bobot += 4
        elif nilai == "B":
            bobot += 3
        elif nilai == "C":
            bobot += 2
        elif nilai == "D":
            bobot += 1
        else:
            nilai += 0
    total = bobot / x
    print(f"Nilai IPS anda semester ini {total:.2f}")
print("Program perhitungan IPS mahasiswa")
ips(x = int(input("Berapa jumlah mata kuliah? ")))
```

Gambar 6.5: Program penghitung IPS mahasiswa.

Fungsi bernama ips yang dirancang untuk menghitung Indeks Prestasi Semester (IPS) mahasiswa berdasarkan jumlah mata kuliah yang diambil. Di awal program, pengguna diminta untuk menginput jumlah mata kuliah (x), yang kemudian digunakan sebagai batas perulangan for. Di dalam perulangan tersebut, program akan meminta input nilai huruf untuk setiap mata kuliah secara berurutan, lalu mengonversinya menjadi poin bobot numerik menggunakan struktur percabangan if-elif-else: nilai "A" berbobot 4, "B" berbobot 3, "C" berbobot 2, dan "D" berbobot 1.

Setelah semua nilai mata kuliah diinput dan bobotnya dijumlahkan ke dalam variabel bobot, program menghitung nilai rata-rata dengan membagi total bobot tersebut dengan jumlah mata kuliah (x). Hasil akhirnya kemudian ditampilkan ke layar dengan format dua angka di belakang desimal (.2f). Sebagai catatan kecil, terdapat sedikit kesalahan logika pada bagian else (ketika nilai selain A-D dimasukkan), di mana variabel nilai yang bertipe teks coba ditambah dengan angka 0, yang secara teknis akan menyebabkan error jika dijalankan.